THE MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION

ITMO University
(ITMO)

Faculty of Control Systems and Robotics

Task 4
for the subject
"Simulation of Robotic Systems"

Student:
Group No. R4137c *Mughal, Umer Ahmed Baig*
ITMO ID. 472268

Tutor:
Professor *Borisov, Ivan*

Assistant *Rakshin, Egor*

Saint Petersburg 2024

CONTENTS

0.1 Objective 1
0.2 Tasks 1

1 XML MODEL OF CLOSED LOOP KINEMATIC MECHANISM 2
1.1 Optimus Mechanism Overview 2
1.2 XML Model of Optimus Knee Mechanism 2
1.3 Structural Integrity through Kinematic Constraints 3
1.4 Driving the Mechanism with an Actuator 3

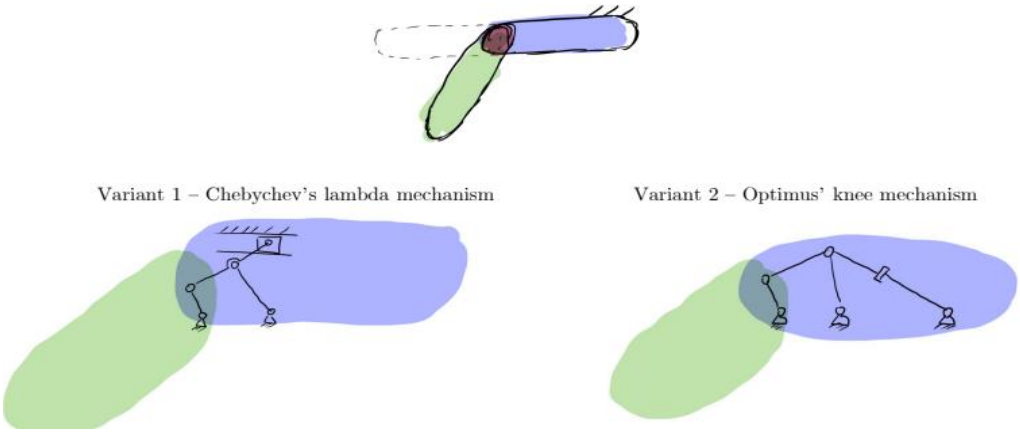
2 PYTHON SCRIPT FOR XML MODEL SIMULATION OF OPTIMUS KNEE MECHANISM 5
2.1 Initialization and Defining Parameters 5
2.2 Controller and Simulation Loop 5
2.3 Visualization and Data Collection 6
2.4 Visualizing Simulation Results through Graphs 7

3 CONCLUSION 9

TASK DESCRIPTION

0.1 Objective:

The objective of the work is to model a closed kinematics mechanism for knee motion.



0.2 Tasks:

1. Build a closed-loop kinematic mechanism in an XML file using the option from the table.
2. Set mechanism in motion by adding a linear motor.
3. Plot the rotation of the crank OA as a function of the linear displacement of the linear actuator.

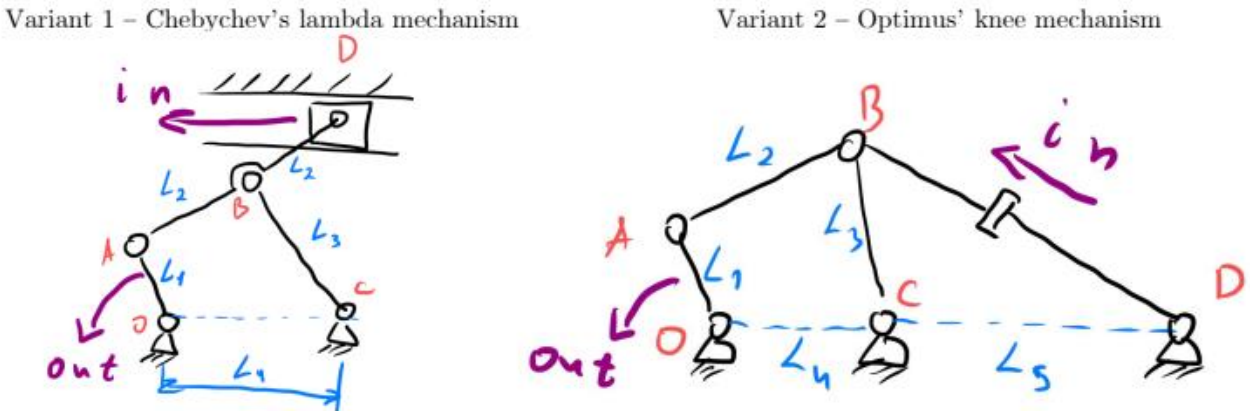


Figure 1: - XML Model for Closed Kinematics Mechanism for Knee Motion.

The initial data of variant 2 for completing the task are presented in Table 1.

Table 1

Variant 2	L1	L2	L3	L4	L5
Optimus Mechanism	0.059	0.0767	0.0885	0.059	0.295

Given values of the parameters according to the task 4.

1 XML MODEL OF CLOSED LOOP KINEMATIC MECHANISM

This report explains the simulation model of the Optimus Knee Mechanism, designed using MuJoCo XML. The mechanism aims to replicate the motion of a knee joint using a combination of links, joints, and actuator. Its hierarchical design and precise control through a linear actuator enable an accurate representation of kinematic behavior. The model simulates the knee's motion by incorporating rotational joints and actuator within a dynamic environment.

1.1 Optimus Mechanism Overview

The Optimus mechanism models a simplified yet functional knee joint. It includes three main links, each connected by rotational joints. These joints allow motion within predefined limits, simulating the realistic bending and extension of a knee. The integration of an actuator makes the mechanism capable of performing controlled movements, replicating the operation of a robotic or biomechanical knee.

1.2 XML Model of Optimus Knee Mechanism

The structure of the Optimus knee mechanism is defined using a hierarchical arrangement of links and joints. The links (Length_1, Length_2, and Length_3) represent mechanical components of the knee, connected by hinge joints (O, A, B, and C) to allow rotational motion. Each joint specifies attributes such as axis, range of motion, and damping, ensuring realistic simulation of joint mechanics. The components are visually represented using geometries such as cylinders and boxes, with attributes like size, position, and color for clarity. A floor plane is added for stability, and light sources enhance visualization.

The below code defines the initial settings for the simulation environment in MuJoCo, including the time step and the spatial statistics of the model. It also specifies the visual aspects of the simulation, such as lighting and textures, to enhance the realism of the 3D model.

```
<mujoco model="Optimus_Knee_Mechanism_472268">
    <option timestep="0.001" />

    <statistic center="0.15 0 0.5" extent="0.5" />

    <visual>
        <headlight diffuse="0.6 0.6 0.6" ambient="0.25 0.25 0.25" />
        <rgba haze="0.05 0.1 0.15 1" />
        <global azimuth="85" elevation="-10" />
    </visual>

    <asset>
        <texture type="skybox" builtin="gradient" rgb1="0.3 0.5 0.7" rgb2="0.1 0.2 0.3" width="256"
            height="256" />
        <texture type="2d" name="groundplane" builtin="checker" mark="edge" rgb1="0.5 0.6 0.7"
            rgb2="0.005 0.005 0.005" markrgb="0.8 0.8 0.8" width="300" height="300" />
        <material name="groundplane" texture="groundplane" texuniform="true" texrepeat="10 10"
            reflectance="0.2" />
    </asset>
```

To achieve the stated objective, the XML model of the Optimus Knee Mechanism has been developed. The following code was compiled to create the XML model, defining the system's structure, and kinematics to simulate the knee's motion in a dynamic environment.

```
<worldbody>
    <light pos="0 0 1.5" />
    <light pos="0 0 3" dir="0 0 -1" directional="true" />
    <geom name="floor" size="0 0 .125" type="plane" material="groundplane" conaffinity="15" condim="3" />
    <body pos="0 0 0.5" euler="0 -10.5 0">
        <joint name="O" axis="0 1 0" range="-80 80" damping="0.1" />
        <geom name="O" type="cylinder" pos="0 0 0" size="0.007 0.007" rgba="0.5 0.5 0.5 1" euler="90 0 0" contype="0" />
        <geom name="Length_1" type="cylinder" pos="0 0 0.0295" size="0.003 0.0295" rgba="1 0 0 1" contype="0" />
        <body pos="0 0 0.059" euler="0 17 0">
            <joint name="A" axis="0 1 0" range="-35 180" />
            <geom name="A" type="cylinder" pos="0 0 0" size="0.007 0.007" rgba="0.5 0.5 0.5 1" euler="90 0 0" contype="0" />
            <geom name="Length_2" type="cylinder" pos="0.038 0 0.023" size="0.003 0.038" euler="0 60 0" rgba="0 1 0 1" contype="0" />
            <site name="s1" pos="0.0767 0 0.04" size="0.0025" />
        </body>
    </body>

    <body pos="0.072 0 0.5">
        <joint name="C" axis="0 1 0" />
        <geom name="C" type="cylinder" pos="0 0 0" size="0.007 0.007" rgba="0.5 0.5 0.5 1" euler="90 0 0" contype="0" />
        <geom name="Length_3" type="cylinder" size="0.003" fromto="0 0 0 0 0 0.0885" rgba="0 0 1 1" contype="0" />
        <site name="s2" pos="0 0 0.085" size="0.0025" />
        <body pos="0 0 0.0885">
            <joint name="B" axis="0 1 0" range="-35 180" />
            <geom name="B" type="cylinder" pos="0 0 0" size="0.0085 0.0085" rgba="0.5 0.5 0.5 1" euler="90 0 0" contype="0" />
        </body>
    </body>

    <body name="Input" pos="0.25 0 0.5" euler="0 -75 0">
        <geom type="box" size="0.01 0.01 0.02" pos="0.025 0 -0.055" rgba="0.1 0.1 0.1 1" />
        <site name="LA_site" pos="0.025 0 -0.055" size="0.005 0.005" rgba="0.5 0.5 0.5 1" />
    </body>

</worldbody>
```

1.3 Structural Integrity through Kinematic Constraints

To ensure the model functions as a cohesive system, constraints are applied to specific points in the mechanism. These constraints, defined in the equality section, use site connections to link the ends of links (s1 and s2). This ensures proper alignment and maintains the structural integrity of the mechanism. These connections prevent unphysical behavior, allowing the simulated links to move realistically according to the specified degrees of freedom.

```
<equality>
    <connect site1="s1" site2="s2" />
</equality>
```

1.4 Driving the Mechanism with an Actuator

The movement of the knee mechanism is driven by a motor actuator integrated into the model. The actuator is linked to the LA_site, providing controlled motion through linear displacement. Key parameters, such as the control range (0 to 0.001) and scaling factor (gear), are configured to regulate the actuator’s force and ensure smooth operation. The motor drives the crank, translating linear motion into rotational movement at the knee joints. This feature allows precise control, which is crucial for simulating realistic joint dynamics.

```
<actuator>
    <motor name="Linear_Actuator" slidersite="LA_site" cranksite="s2" cranklength="0.295" ctrlrange="0 0.001" gear="1" />
</actuator>
```

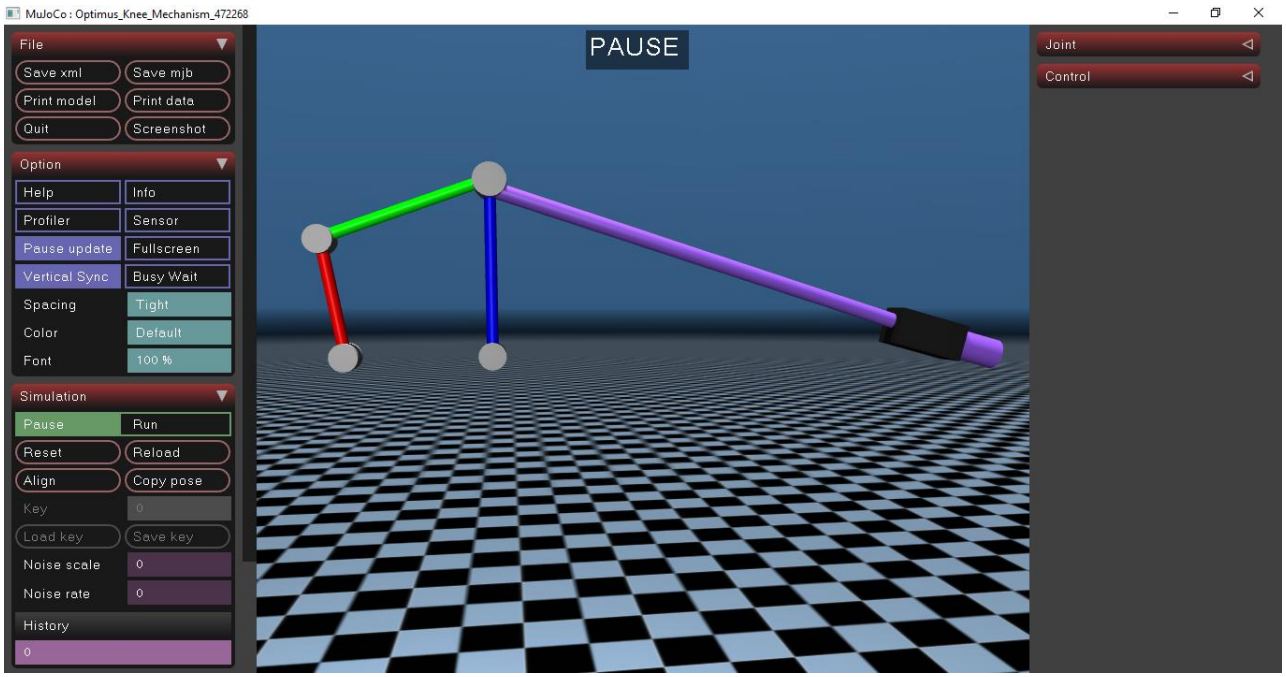


Figure 2: XML Model of Optimus Knee Mechanism

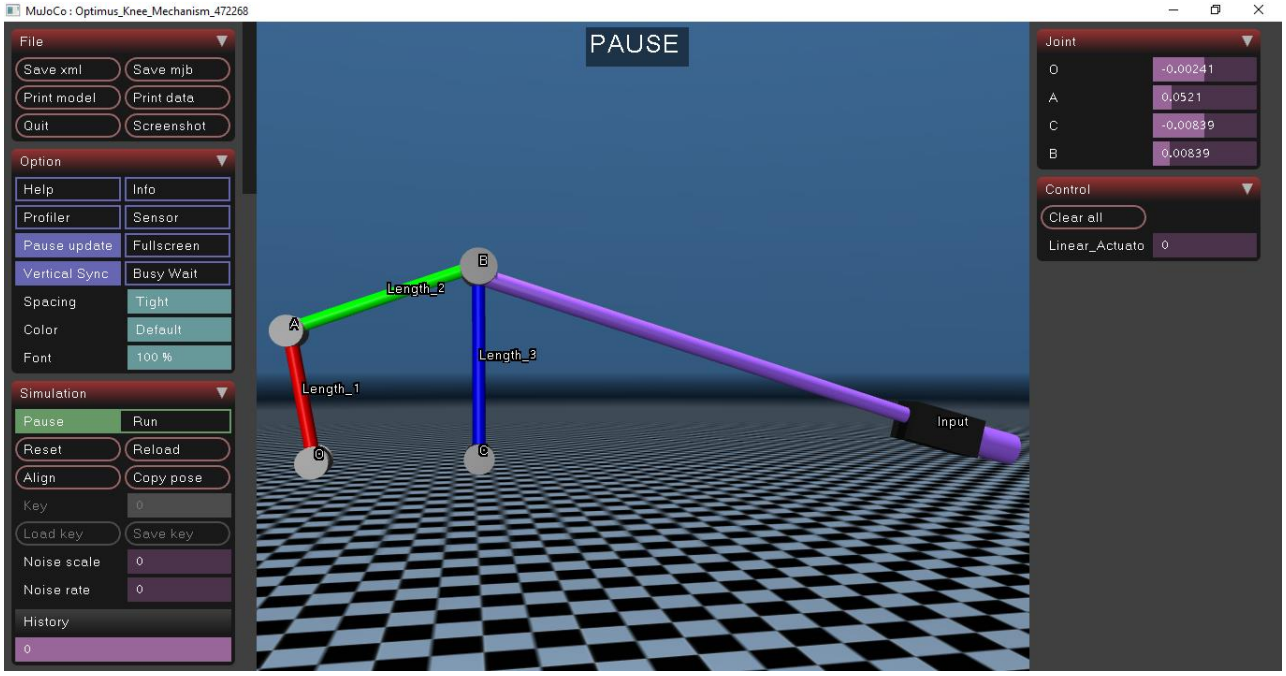


Figure 3: XML Model of Optimus Knee Mechanism with Joints and Actuator.

2 PYTHON SCRIPT FOR XML MODEL SIMULATION OF OPTIMUS KNEE MECHANISM

The simulation model of the Optimus Knee Mechanism is implemented using the MuJoCo. The system simulates the motion of the knee joint using linear actuator and rotational joints, and tracks the kinematic behavior through various parameters such as actuator positions and angle deviations. The following Python script controls the actuator, runs the simulation, and visualizes the resulting data.

2.1 Initialization and Defining Parameters

Firstly, import the necessary libraries for simulation, visualization, numerical operations, and rotation computations. The MuJoCo XML model is loaded using the specified file path. The simulation begins by loading the XML model of the Optimus Knee Mechanism and initializing necessary parameters, including maximum control values and simulation duration. The control parameters define the range and step increments for the actuator control, allowing for fine adjustments in the simulation's execution. The initial actuator position and Euler angles are recorded to track deviations throughout the simulation.

2.2 Controller and Simulation Loop

A controller function is implemented to gradually increase the actuator's control input, simulating the movement of the knee mechanism over time. The loop runs for the predefined simulation duration, calculating and recording the actuator position and angular deviations. These values are then used to monitor how the actuator influences the system's kinematics. The ‘**controller()**’ function updates the actuator control signal by incrementing it until it reaches its maximum value. The ‘**simulate()**’ function manages the main simulation, stepping through each cycle and collecting data for the actuator position and Euler angles.

- **Function Details**

1. **get_euler_angles_from_rotation_matrix(rotation_matrix):** This function converts a given rotation matrix into Euler angles using SciPy’s Rotation class, allowing the rotation of geometries to be tracked in 3D space.

2. **get_l1_rotation(model, data, geom_name):** This function calculates the Euler angles of a specific geometry in the model (e.g., the "L1" link). It retrieves the rotation matrix of the geometry and converts it to Euler angles, which are used to track angular deviations during the simulation.

2.3 Visualization and Data Collection

The simulation runs in real-time using MuJoCo's passive viewer for rendering the model's movements. Actuator positions and angle deviations are recorded and plotted for analysis, providing a visual representation of the relationship between actuator displacement and the resulting angular deviation.

```python
import numpy as np
import time
import mujoco
import mujoco.viewer
import matplotlib.pyplot as plt
from scipy.spatial.transform import Rotation as R

# Path to your XML model file
MODEL_NAME = 'C:\\Users\\ADMIN\\Desktop\\Task_4.xml'

# Constants for simulation
MAX_CTRL = 0.01  # Maximum actuator control range
CTRL_INCREMENT = 0.0005  # Step increment for actuator control
SIMULATION_TIME = 5  # Total simulation duration in seconds

# Function to get Euler angles from a rotation matrix
def get_euler_angles_from_rotation_matrix(rotation_matrix):
    rotations = R.from_matrix(rotation_matrix)
    euler = rotations.as_euler('xyz', degrees=True)
    return euler

# Function to calculate the rotation of a given geometry
def get_l1_rotation(model, data, geom_name):
    geom_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_GEOM, geom_name)
    rotation_matrix = np.reshape(data.geom_xmat[geom_id], (3, 3))
    rotation = R.from_matrix(rotation_matrix)
    euler_angles = rotation.as_euler('xyz', degrees=True)
    return euler_angles
# Function to control the actuator
def controller(model, data, current_ctrl):
    if current_ctrl < model.actuator_ctrlrange[0][1]:
        current_ctrl += CTRL_INCREMENT
    else:
        current_ctrl = model.actuator_ctrlrange[0][1]
    data.ctrl[0] = current_ctrl
    return current_ctrl

# Main simulation function
def simulate():
    # Load the model and initialize data
    model = mujoco.MjModel.from_xml_path(MODEL_NAME)
    data = mujoco.MjData(model)

    # Initialize actuator and angle tracking
    actuator_positions = []
    angle_deviations = []

    # Initialize initial states
    initial_actuator_position = data.actuator_length[0]
    initial_euler_angles = get_l1_rotation(model, data, 'Length_1')
    initial_angle_y = initial_euler_angles[1]

    # Debug: Print initial state
    print("Initial actuator position:", initial_actuator_position)
    print("Initial Y angle:", initial_angle_y)

    # Open the viewer for rendering
    with mujoco.viewer.launch_passive(model, data) as viewer:
        start_time = time.time()
        current_ctrl = model.actuator_ctrlrange[0][0]  # Start at the minimum control value
```

```python
        # Run the simulation loop
        while viewer.is_running() and time.time() - start_time < SIMULATION_TIME:
            step_start = time.time()

            # Calculate the current rotation and angle deviation
            euler_angles = get_l1_rotation(model, data, 'Length_1')
            current_angle_y = euler_angles[1] - initial_angle_y  # Deviation from initial angle
            angle_deviations.append(current_angle_y)

            # Calculate actuator position deviation
            current_actuator_position = data.actuator_length[0]
            actuator_deviation = current_actuator_position - initial_actuator_position
            actuator_positions.append(actuator_deviation)

            # Apply controller logic
            current_ctrl = controller(model, data, current_ctrl)

            # Step the simulation and render
            mujoco.mj_step(model, data)
            viewer.sync()

            # Sleep to match real-time simulation step
            next_step = model.opt.timestep - (time.time() - step_start)
            if next_step > 0:
                time.sleep(next_step)
    # Plot the results
    plt.figure()
    plt.plot(actuator_positions, angle_deviations, label="Angle deviation vs actuator position", color="purple")
    plt.xlabel("Actuator Deviation")
    plt.ylabel("Y Angle Deviation (degrees)")
    plt.title("Length_1 Angle Deviation vs Actuator Position")
    plt.grid(True)
    plt.legend()
    plt.show()

# Entry point
if __name__ == '__main__':
    simulate()
```

2.4 Visualizing Simulation Results through Graphs

The above code generates a simulation graph that visualizes the relationship between the actuator's movement and the rotation of the OA link (Length_1). As the actuator moves, the angle of the OA link (Length_1) is plotted against the actuator's position, revealing how the actuator's motion influences the link's rotation. The graph helps illustrate the dynamic response of the system, which may not follow a linear pattern, depending on the mechanism's design. Additionally, the graph tracks the motion trajectories, highlighting changes in the system's behavior over time, and providing insights into the actuator's effect on the rotational movement of the OA link (Length_1).

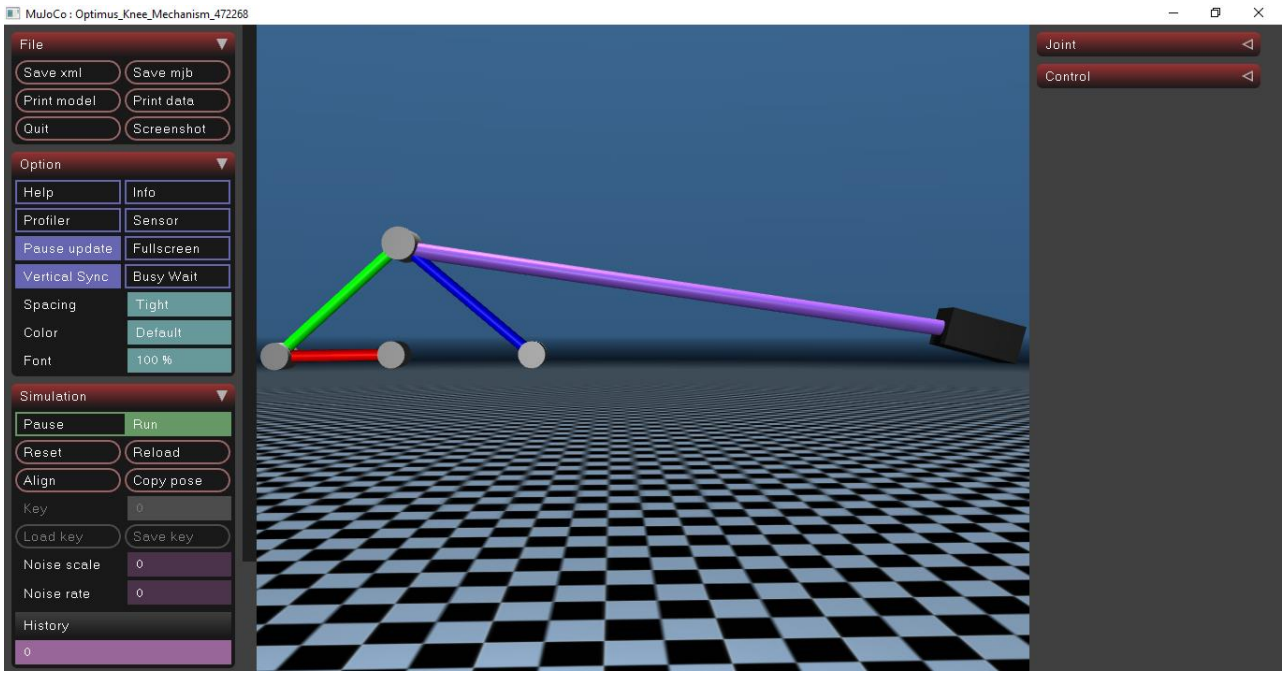


Figure 4: XML Model Simulation illustrates the rotation of the OA link (Length_1) based on actuator movement.

The graph generated from python code provides insight into how effectively the actuator's movement translates into rotational motion. A steep curve indicates that small changes in actuator position lead to significant rotation, reflecting a highly responsive system. In contrast, a flatter curve suggests that the system is less sensitive and responds more slowly to actuator adjustments. This analysis helps in understanding the mechanism's dynamic performance and optimizing its control for accurate motion. The plot tracks the relationship between actuator position and Y angle deviation, ranging from -10 to -90 degrees, offering a clear view of the actuator's impact on rotation during the simulation.

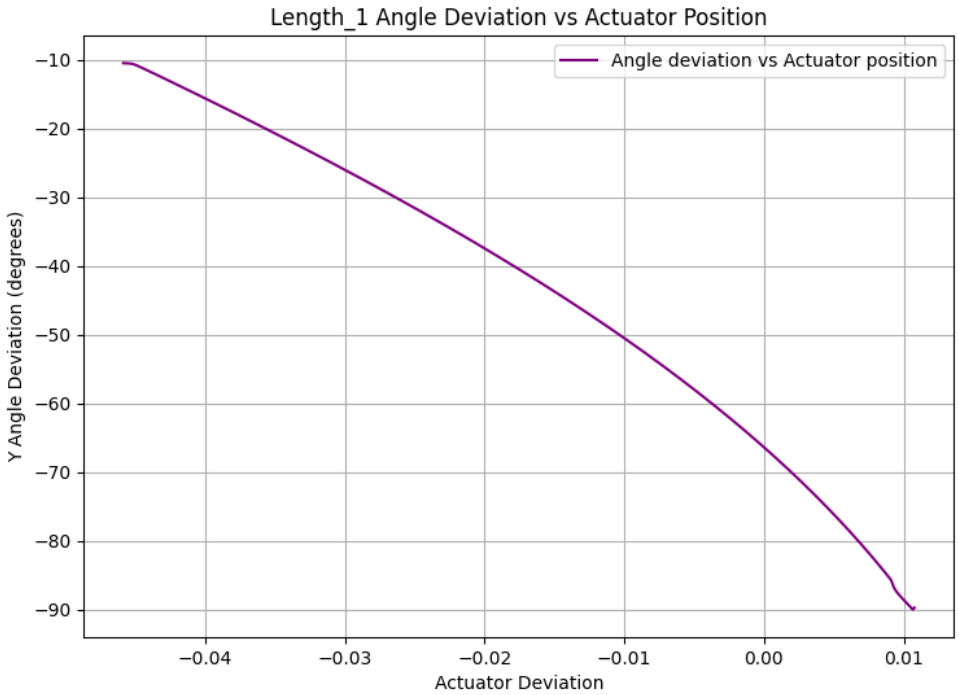


Figure 5: Actuator position and Angle deviation of OA Link Length_1 over time.

3 CONCLUSION

The simulation effectively demonstrates the kinematic behavior of the Optimus Knee Mechanism using MuJoCo, focusing on the relationship between actuator position and the rotation of the OA link (Length_1). By incrementally controlling the actuator, the system's response is tracked through angular deviations. The plotted data reveals how actuator movements influence the rotation, with steep curves indicating high sensitivity and flatter curves suggesting slower responses. This analysis provides valuable insights into the system's dynamic performance, helping optimize actuator control for more precise knee joint motion.